

인공지능수학 (2026-1)

11. 경사 하강법

국립금오공과대학교 컴퓨터공학부
컴퓨터공학전공 / 인공지능공학전공

김 경 수

목차



- I. 경사 하강법 개요
- II. 경사 하강법의 구현
- III. 경사 하강법의 수렴성 분석

- ① 경사 하강법의 개요와 동작 원리를 이해하고, 경사 하강법을 통해 최적해를 탐색하는 일련의 과정을 논리적으로 설명할 수 있다.
- ② 실제 프로그래밍 언어를 이용하여 경사 하강법 알고리즘을 구현하고, 이를 이용하여 간단한 함수의 해를 계산할 수 있다.
- ③ 경사 하강법의 수렴성(convergence)을 이론적으로 증명하는 방법을 이해하고 그 과정을 논리적으로 설명할 수 있다.

목적 함수와 파라미터

- 목적 함수(objective function) f 가 다음과 같이 주어진다고 가정하자.

$$f(\vec{x}; \theta_1, \dots, \theta_n) \in \mathbb{R}$$

- 목적 함수의 구성

- 입력 : $\vec{x} = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^{n \times 1}$

- 출력 : $f(\vec{x}; \theta_1, \dots, \theta_n) \in \mathbb{R}$

- 파라미터 : $\theta_1, \theta_2, \dots, \theta_n$

- 우리의 목표

- 함수의 출력 값(= 적합도: fitness)을 최소로 만드는 최적의 해 $\vec{x}^*$ 를 찾는 것이다.

- 즉, $\vec{x}^* = \arg \min_{\vec{x}} f(\vec{x}; \theta_1, \dots, \theta_n)$.

I. 경사 하강법 개요

분석적 방법 vs. 수치적 방법

- 주어진 목적 함수의 최적해를 구하기 위한 방법
 - **분석적(해석적) 방법:** 주어진 수식을 직접적으로 분석해서 해를 구하는 방법으로 우리가 학교에서 배운 다양한 수학 공식을 이용하여 해를 구하는 것이 대표적인 방법
 - **수치적 방법:** 탐색 알고리즘을 활용하여 반복적으로 해를 탐색함으로써 최적의 해를 발견하는 방법
- 분석적 방법과 수치적 방법의 예: 이차 방정식 $y = x^2 - 6x + 9$ 를 계산하는 방법
 - 분석적 방법 → 근의 공식 또는 완전 제곱식을 이용하여 해를 구한다.
 - 수치적 방법 → x 값을 임의의 값으로 초기화 한 후 경사 하강법을 이용하여 반복적 탐색을 수행함으로써 해를 구한다.

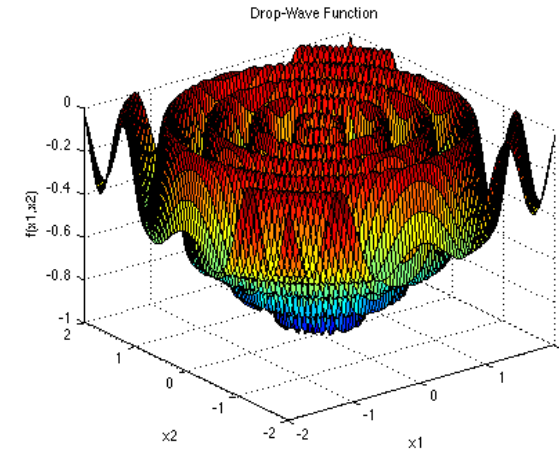
분석적 방법 vs. 수치적 방법

- 수치적 방법을 사용하는 이유

- 기존의 분석적 방법은 복잡한 목적 함수의 최적해를 계산하는 데 한계가 있다.
- 특히, 복잡한 목적 함수일수록 우리가 배운 수학 공식을 적용하여 정확한 최적해를 계산하는 것이 매우 어려운 경우가 대다수이다.

$$(예) f(x_1, x_2) = -\frac{1 + \cos\left(12\sqrt{x_1^2 + x_2^2}\right)}{0.5(x_1^2 + x_2^2) + 2}$$

where $x_1, x_2 \in [-5.12, 5.12]$



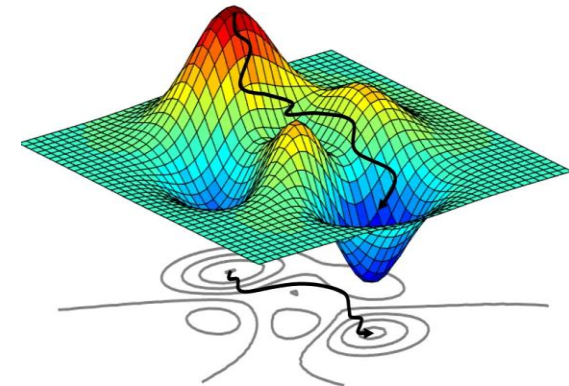
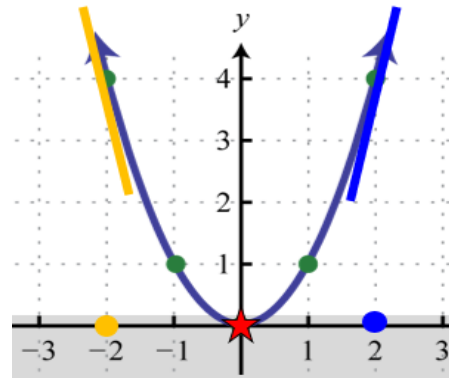
- 이처럼, 수학 공식을 적용하여 해를 구할 수 없는 경우에는 수치적 방법을 동원하여 정확한 최적해 대신 **근사 최적해(approximate optimal solution)**를 구하는 전략이 필요하다.

경사 하강법(gradient descent)이란?

- 경사 하강법은 목적 함수의 근사 최적해를 구하기 위한 수치적 방법 중 하나
- 반복적인 해 탐색을 통해 목적 함수를 최소화(or 최대화)할 수 있는 최적해를 발견하는 것을 목표로 한다.
- 경사 하강법의 아이디어
 - 기본적으로 야간에 산속에서 가장 낮은 곳으로 내려오는 방법과 유사하다.
 - 즉, 경사가 급한 방향으로 내려갈 수록, 평지에 빠르게 도착할 수 있다.
 - 한편, 경사가 완만한 방향으로 내려갈 수록, 평지에 천천히 도착하게 된다.
 - 즉, 현재 위치를 기준으로 경사(기울기)를 파악하는 것이 필요하다.

경사 하강법(gradient descent)이란?

- 현재 해를 기준으로 기울기를 계산한 후, 기울기에 따라 다음 해를 탐색한다.
 - 아래 그림에서 산의 지형(사면) → 목적 함수(objective function)
 - 현재 / 다음 위치 → 현재 / 다음 시점의 최적해(solutions at current / next steps)
 - 현재 위치에서의 경사도 → 현재 최적해에서 함수의 기울기(그레디언트: gradient)



- 매 단계에서 현재 최적해를 기준으로 함수의 기울기를 계산해야 하므로, 경사 하강법에서 다루는 목적 함수는 **미분 가능한 함수(differentiable function)**여야 한다.

경사 하강법의 수행 절차

- ① 함수 $f(x)$ 의 최적해를 구하기 위해 변수 x 를 임의의 초기값 $x^{(1)}$ 으로 설정한다.
- ② 현재 시점 t 에서의 변수 $x^{(t)}$ 에 대한 함수의 기울기를 파악하기 위해 도함수 $\nabla_x f(x^{(t)})$ 를 계산한다.
- ③ 계산된 기울기를 이용해 현재 시점에서의 변수 $x^{(t)}$ 를 다음 시점에서의 변수 값 $x^{(t+1)}$ 로 갱신한다. (최적해 업데이트)

$$x^{(t+1)} = x^{(t)} - \eta \nabla_x f(x^{(t)})$$

- ④ 현재 시점 t 를 $t + 1$ 로 증가시킨다.
- ⑤ 수렴 또는 종료 조건을 확인하고, 조건을 충족하지 않으면 ② ~ ⑤ 단계를 반복한다.
- ⑥ 상기 ② ~ ⑤의 반복 과정을 통해서 발견된 변수 x 의 최종 값을 함수 f 의 최적해로 판단하고, 이를 반환한다.

Algorithm gradient_descent

$t \leftarrow 1$

$x^{(1)} \leftarrow$ random initial value

while (is_converged() = False) **do**

$g^{(t)} \leftarrow \nabla_x f(x^{(t)})$

$x^{(t+1)} \leftarrow x^{(t)} - \eta g^{(t)}$

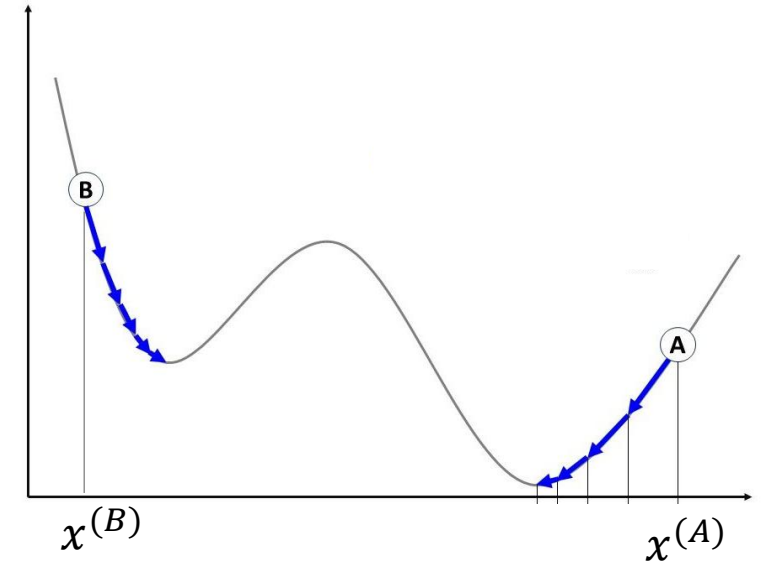
$t \leftarrow t + 1$

end

return $x^{(t)}$

초기화(initialization)

- 경사 하강법 알고리즘을 시작할 지점인 초기 해(initial solution)를 설정한다.
- 초기 해를 어떻게 설정하는가에 따라 최적해 탐색 성능에 영향을 미칠 수 있다.
 - 왼쪽 그림에서 초기 해를 $x^{(A)}$ 로 설정한 경우와 $x^{(B)}$ 로 설정한 경우 탐색 과정에서 도달하는 최적해의 지점이 달라지게 된다.
 - 왼쪽 예제에서는 초기 해를 $x^{(A)}$ 로 설정했을 때 전역 최적해로 수렴하게 된다.
- 따라서, 해를 초기화 할 때에는 일반적으로 임의의 값으로 랜덤하게 초기화하는 방법이 널리 활용된다.



목적 함수에 대한 도함수의 계산

- 경사 하강법에서 최적해를 업데이트하기 위해서는 현재 최적해에서 목적 함수의 도함수가 활용된다.

$$x^{(t+1)} = x^{(t)} - \eta \nabla_x f(x^{(t)})$$

- 목적함수에 대한 미분
 - 경사 하강법의 핵심 원리는 현재 최적해를 기준으로 목적 함수의 기울기(경사)를 계산하고, 목적 함수의 기울기에 따라 최적해의 업데이트 정도를 결정하는 것이다.
 - 이를 위해, 현재 최적해를 기준으로 목적 함수의 미분 계수(= 도함수)를 계산한다.
- 기호 $\nabla_x f(x)$
 - 주어진 목적 함수 $f(x)$ 를 x 에 대하여 미분한 도함수를 의미한다.

목적 함수에 대한 도함수의 계산

- 현재 탐색하는 최적해에 대한 변수로 주어진 목적 함수의 도함수를 계산한다.
- 이를 위해, 목적 함수가 다변수 함수인 경우 **편미분(partial differential)**을 수행한다.
- 편미분의 정의

➤ 함수 $f : \mathbb{R}^n \rightarrow \mathbb{R}$ 에 대하여, 변수 x_i 에 대한 편미분은 다음과 같은 극한으로 정의된다.

$$\frac{\partial}{\partial x_i} f(x_1, \dots, x_n) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - f(x_1, \dots, x_{i-1}, x_i, x_{i+1}, \dots, x_n)}{h}$$

➤ 즉, x_i 에 대한 편미분은 다변수 함수에서 미분하려는 변수에 대한 순간 변화율만을 고려한다.

목적 함수에 대한 도함수의 계산

- 편미분을 쉽게 수행하는 방법
 - 현재 미분하려는 변수를 제외한 나머지 모든 변수는 상수(constant)로 취급한다.
 - 나머지 미분하는 방법은 기존의 미분법과 동일하다.

- [예제 1] 아래 목적 함수를 변수 x_2 에 대하여 편미분하시오.

$$f(x_1, x_2, x_3, x_4, x_5) = x_1 x_2^2 + e^{x_3 x_4} + \ln x_5$$

- [풀이]

$$\begin{aligned}\frac{\partial}{\partial x_2} f(x_1, x_2, x_3, x_4, x_5) &= \frac{\partial}{\partial x_2} x_1 x_2^2 + \frac{\partial}{\partial x_2} e^{x_3 x_4} + \frac{\partial}{\partial x_2} \ln x_5 \\ &= 2x_1 x_2\end{aligned}$$

목적 함수에 대한 도함수의 계산

- [예제 2] 아래 목적 함수를 변수 x_1 에 대하여 편미분하시오.

$$f(x_1, x_2, x_3, x_4, x_5) = \sin(x_1 x_2) + x_3^2 x_4 + (x_5)^{-1}$$

- [풀이]

$$\begin{aligned}\frac{\partial}{\partial x_1} f(x_1, x_2, x_3, x_4, x_5) &= \frac{\partial}{\partial x_1} \sin(x_1 x_2) + \frac{\partial}{\partial x_1} x_3^2 x_4 + \frac{\partial}{\partial x_1} \left(\frac{1}{x_5} \right) \\ &= \cos(x_1 x_2) x_2\end{aligned}$$

목적 함수에 대한 도함수의 계산

- [예제 3] 아래 목적 함수를 변수 x_1 에 대하여 편미분하시오.

$$f(x_1, x_2, x_3, x_4, x_5) = \ln(x_1^2 + e^{x_2 x_3} \sin x_4) + \frac{x_5}{x_1 x_2}$$

- [풀이]

- 첫 번째 항 $\ln(x_1^2 + e^{x_2 x_3} \sin x_4)$ 을 편미분하기 위해 $x_1^2 + e^{x_2 x_3} \sin x_4 = u$ 라 하자. 그렇다면,

$$\frac{\partial}{\partial x_1} \ln u = \frac{1}{u} \cdot \frac{\partial u}{\partial x_1} = \frac{1}{u} \cdot 2x_1 = \frac{2x_1}{x_1^2 + e^{x_2 x_3} \sin x_4}$$

- 두 번째 항 $\frac{x_5}{x_1 x_2}$ 을 편미분하면 다음과 같다.

$$\frac{\partial}{\partial x_1} \left(\frac{x_5}{x_1 x_2} \right) = \frac{-x_5}{x_1^2 x_2}$$

- 따라서, $\frac{\partial}{\partial x_1} f(x_1, x_2, x_3, x_4, x_5) = \frac{2x_1}{x_1^2 + e^{x_2 x_3} \sin x_4} - \frac{x_5}{x_1^2 x_2}$ 이다.

최적해의 업데이트

- 최적해의 업데이트 방법

$$x^{(t+1)} = x^{(t)} - \eta \nabla_x f(x^{(t)})$$

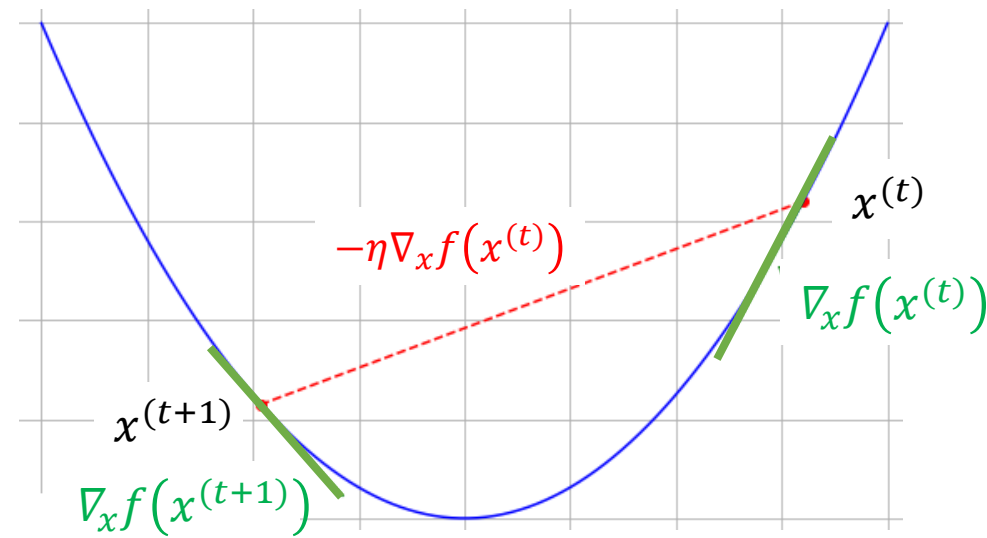
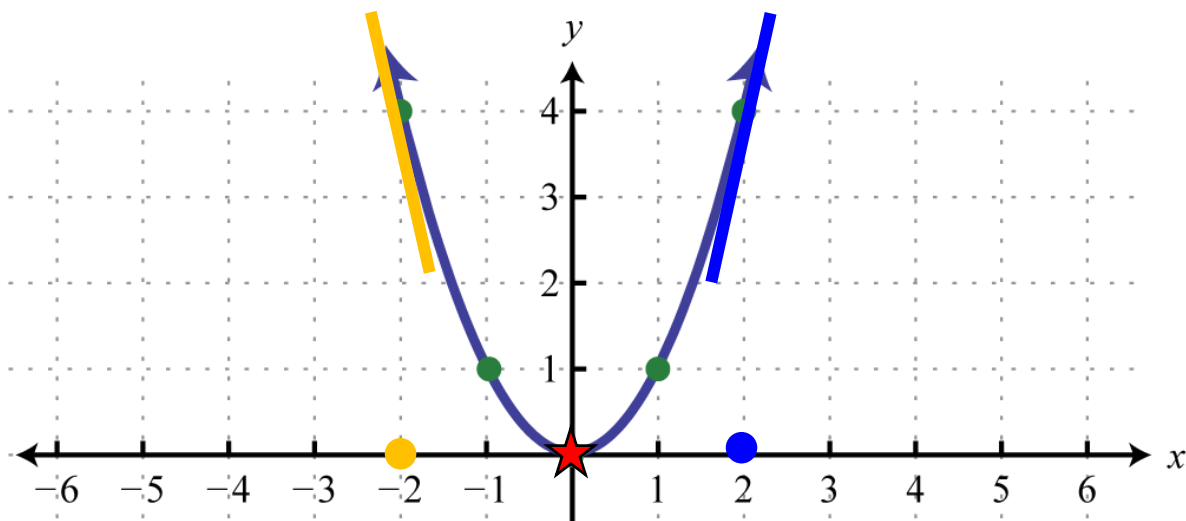
- 현재 시점의 해 $x^{(t)}$ 를 기준으로, 목적 함수 $f(x^{(t)})$ 의 도함수 $\nabla_x f(x^{(t)})$ 계산이 완료되면, 이 도함수의 크기에 비례하여 최적해 $x^{(t)}$ 를 업데이트한다.
- 이때, 아래의 원리에 따라 최적해를 업데이트한다.
 - ① 경사가 급할 수록(= 기울기가 클 수록) 최적해의 변화의 정도가 커져야 한다.
 - ② 반대로, 경사가 완만할 수록(= 기울기가 작을수록) 최적해의 변화의 정도가 작아져야 한다.
 - ③ 동시에, 업데이트의 강도를 결정하는 변수로 학습률(learning rate) η 가 활용하여 업데이트가 과도하게 이루어지는 것을 억제해야 한다.

최적해의 업데이트

- 최적해의 업데이트 방법

$$x^{(t+1)} = x^{(t)} - \eta \nabla_x f(x^{(t)})$$

- 마지막으로, 최적해 업데이트 시 부호는 마이너스(-)임에 유의한다(※ 최소화 문제 기준)
 - 이는, 기울기가 양수인 경우 최적해는 현재 해의 왼쪽에 위치하기 때문이다.
 - 반대로, 현재 해를 기준으로 음의 기울기를 갖는 경우 현재 해의 오른쪽에 최적해가 위치한다.
 - 만약 다루는 문제가 “최대화(maximization)”인 경우, 업데이트 시 부호는 플러스(+)가 된다.



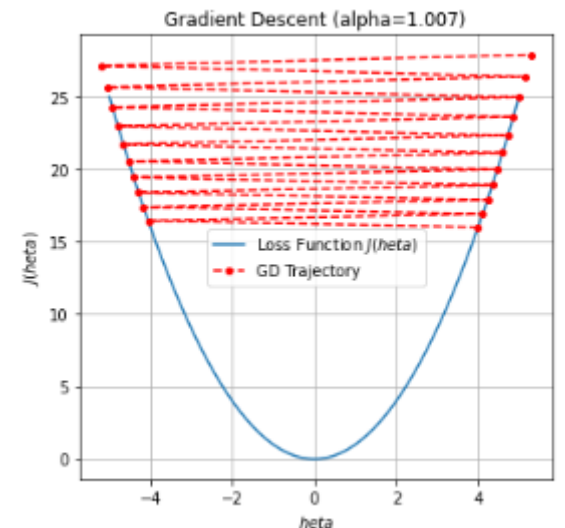
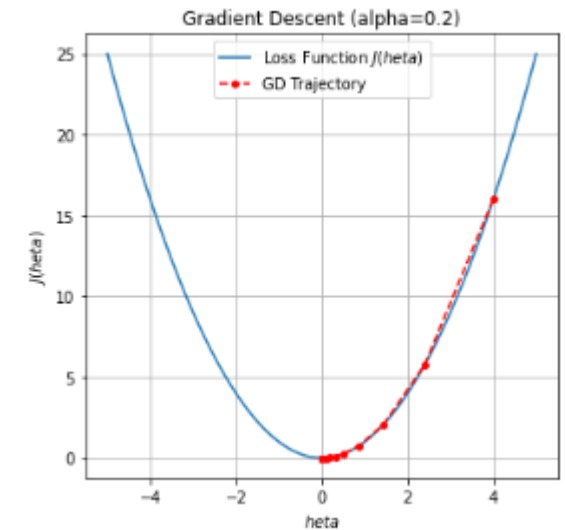
최적해의 업데이트

- 학습률(learning rate) η
 - 목적함수의 기울기 만큼 최적해를 업데이트하는데, 이 때 얼마만큼 최적해를 업데이트할 것인가를 결정하는 변수
 - 목적함수의 도함수에 학습률을 곱하여 최적해의 업데이트 정도를 제어한다.
- 학습률 설정 시 유의사항
 - 설정된 학습률에 따라 발견되는 최적해의 품질이 달라진다.
 - 즉, 학습률의 선택 방법에 따라 최적해 탐색 성능에 긍정적/부정적 영향을 미친다.
 - 최적의 학습률은 경사 하강법을 적용하는 응용(application)에 따라 다르며, 모든 응용에서 최적화된 학습률은 아직까지 발견되지 않았다.

일반적으로 학습률은 작은 값으로 설정하되 반복적인 실험을 통해서 최적의 값을 결정한다.

최적해의 업데이트

- 학습률이 최적해 탐색에 어떠한 영향을 미치는가?
 - 학습률이 너무 작은 경우 (▶ 우측 상단 그림)
 - ✓ 최적해의 업데이트 비율이 매우 작아짐
 - ✓ 이에 따라, 최적해의 수렴 속도가 느려짐
 - 학습률이 너무 큰 경우 (▶ 우측 하단 그림)
 - ✓ 최적해의 업데이트 비율이 매우 커짐
 - ✓ 최적해로 수렴하지 못하고 발산하는 등 해 탐색의 불안정성이 증가함



최적해의 수렴

- 경사 하강법 알고리즘을 수행하는 과정에서 매 단계에서 발견되는 최적해의 변화 정도가 점차적으로 0에 가까워지는 경우, 최적해가 수렴한 것으로 판단한다.

$$\lim_{t \rightarrow \infty} |x^{(t+1)} - x^{(t)}| \approx 0$$

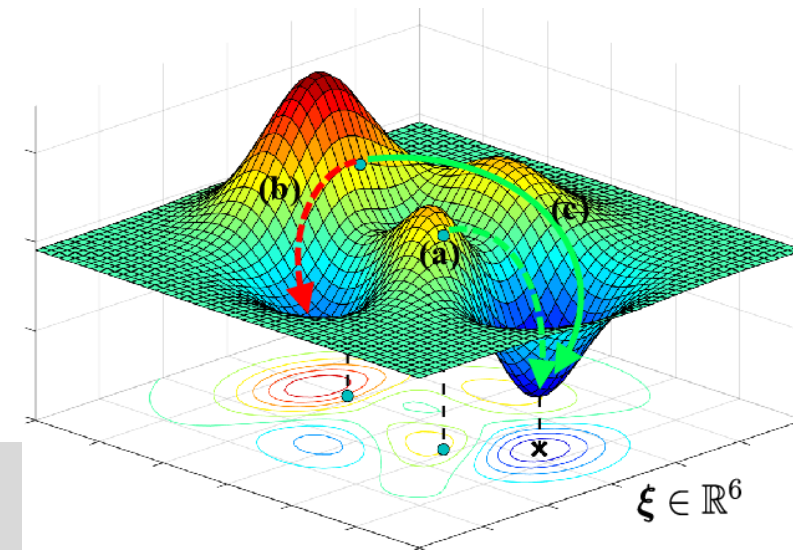
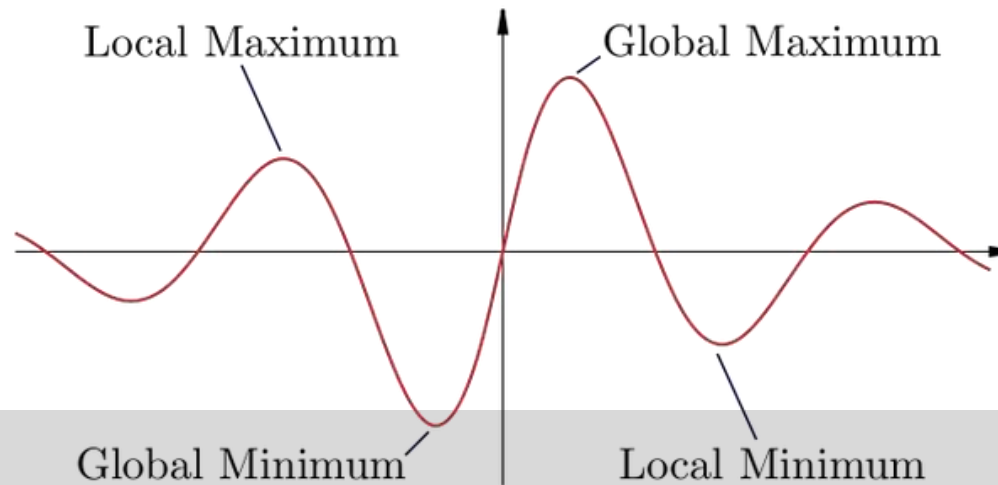
- 즉, 직전에 계산된 최적해와 현재 시점에서 계산된 최적해의 차이가 임계값(threshold value)보다 작은 경우, 해 탐색을 종료한다.
- 즉, 경사 하강법에서 종료 조건은 아래와 같이 구현할 수 있다.

$$\text{if } |x^{(t+1)} - x^{(t)}| < \tau \text{ then break;}$$

- 이 외에도, 최대 탐색 횟수를 지정하고, 이를 초과하는 경우 해 탐색을 종료하는 방법 또한 활용할 수 있다.

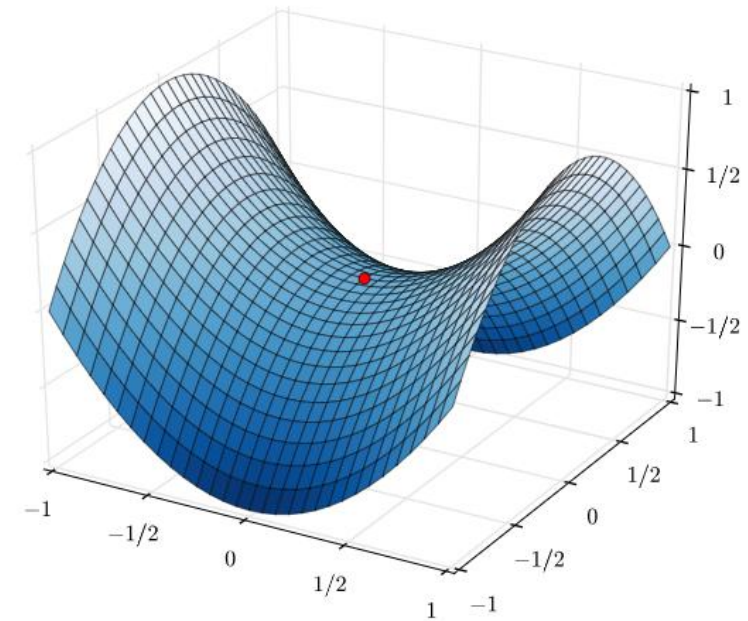
경사 하강법의 주요 이슈(issue)

- 국소 최적해(local minimum or maximum)
 - 경사 하강법에서 다뤄지는 목적 함수들은 대부분 아래와 같이 여러 개의 국소 최적 점을 갖는 복잡한 문제들이다.
 - 경사 하강법으로 해 탐색을 진행할 때, 만약 국소 최적점으로 수렴하는 경우, 여기서 다시 빠져나오기가 매우 어렵다.
 - 이러한 국소 최적점에 빠지게 되면, 더 좋은 적합도를 갖는 최적해로 다시 이동하는 것이 어려우며, 이는 결과적으로 전역 최적해를 발견하는 데 실패하는 주요 요인이 된다.



경사 하강법의 주요 이슈(issue)

- 안장점(saddle point)
 - 특정 축을 기준으로 maximum point이면서 다른 축을 기준으로 minimum point가 되는 지점
 - 만약 최적해 탐색 과정에서 안장점에 도달하게 된다면 국소 최적점과 마찬가지로 그레디언트(경사)만으로 빠져나오기 어렵게 된다.
 - 이러한 안장점에서 빠져나오기 위해서는 기울기의 기울기 (gradient of gradient)를 계산할 수 있어야 하는데, 이는 곧 주어진 목적 함수를 두 번 미분(2차 미분)하는 것과 동일하다.
 - 이에 따라, 2차 미분법에 기반하여 최적해를 탐색하는 다양한 second-order optimization algorithm이 연구되고 있다.



II. 경사 하강법의 구현

경사 하강법 알고리즘의 구현 절차

- ① 주어진 함수의 도함수를 계산하는 함수 구현
- ② 목적 함수 구현
- ③ 수렴 평가 함수 구현
- ④ 경사 하강법 알고리즘 구현
- ⑤ 실제 데이터를 이용한 테스트

① 주어진 함수의 도함수를 계산하는 함수 구현

- 미분의 정의

- 함수 $f(x)$ 의 그래프 위의 두 점 $(x, f(x))$ 와 $(x+h, f(x+h))$ 이 있을 때, 이 두 점이 한없이 가까워질 때 두 점을 잇는 직선의 기울기를 의미한다.
- 즉, h 가 0으로 수렴할 때 두 점 $(x, f(x))$ 와 $(x+h, f(x+h))$ 를 잇는 직선의 기울기이다.

- 상기 정의에 따라 미분 공식(도함수)을 나타내면 다음과 같다.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

① 주어진 함수의 도함수를 계산하는 함수 구현

- 실제 구현에서는 앞에서 설명한 미분 공식의 근사(approximation) 버전인 아래의 함수가 자주 사용된다.
- 아래의 근사식을 **중심 차분 근사(Central Difference Approximation)**라고 한다.

$$f'(x) \cong \lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

① 주어진 함수의 도함수를 계산하는 함수 구현

- [참고] 테일러 급수(Taylor series)

$$\begin{aligned} f(x) &= \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n \\ &= f(a) + f'(a)(x-a) + \frac{1}{2!} f^{(2)}(a)(x-a)^2 + \frac{1}{3!} f^{(3)}(a)(x-a)^3 + \dots \end{aligned}$$

① 주어진 함수의 도함수를 계산하는 함수 구현

- 아래 식이 함수 $f(x)$ 의 도함수 $f'(x)$ 의 근사식인 이유는 다음과 같이 증명할 수 있다.

$$f'(x) \cong \lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

- 함수 $f(x+h)$ 와 $f(x-h)$ 를 테일러 급수(Taylor's series)를 이용하여 근사화하면 다음과 같다.

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + \frac{f^{(3)}(x)}{6}h^3 + \dots$$

$$f(x-h) = f(x) - f'(x)h + \frac{f''(x)}{2}h^2 - \frac{f^{(3)}(x)}{6}h^3 + \dots$$

- 그렇다면, $f(x+h) - f(x-h)$ 는 다음과 같다.

$$f(x+h) - f(x-h) = 2f'(x)h + \frac{2f^{(3)}(x)}{6}h^3 + \dots$$

① 주어진 함수의 도함수를 계산하는 함수 구현

$$f(x+h) - f(x-h) = 2f'(x)h + \frac{2f^{(3)}(x)}{6}h^3 + \dots$$

- 위 수식의 양변을 $2h$ 로 나누면 다음과 같다.

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{f^{(3)}(x)}{3!}h^2 + \dots$$

- 위 수식을 자세히 살펴보면, 실제 도함수인 $f'(x)$ 에 3차 도함수 $f^{(3)}(x)$ 및 그 이상의 항을 오차로 포함하는 근사식이다.
- 이 오차 항은 $h \rightarrow 0$ 일 때 모두 사라지고 최종적으로 원래 도함수인 $f'(x)$ 만 남는다. 즉,

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h} \cong f'(x)$$

- 따라서, 위 공식은 실제 도함수 $f'(x)$ 의 근사 계산 공식이다.

① 주어진 함수의 도함수를 계산하는 함수 구현

- Python에서는 함수 자체를 인자(argument)로 전달하는 함수를 구현할 수 있다.
- 즉, 함수 이름을 입력으로 받아서, 해당 함수에 대한 도함수를 계산하고, 그 결과 (= 도함수)를 반환하는 함수를 구현한다.

```
# Differential of f  
def fdiff(f):  
    def diff(x):  
        h = 1e-8  
        return ( f(x + h) - f(x - h) ) / ( 2 * h )  
  
    return diff
```

② 목적 함수 구현

- 일반적으로 목적 함수는 오차의 정도를 평가하는 오차 함수(또는 손실함수) 또는 최적해를 구하고자 하는 함수를 의미한다.
- 본 실습에서는 간단한 목적 함수로 $f(x) = 3(x - 5)^2 + 2$ 를 구현해보자.

```
# Objective function
def f(x):
    return 3*(x-5)**2 + 2
```

③ 수렴 평가 함수 구현

- 수렴 성능을 평가하는 대표적인 방법으로, 앞에서 언급한 바와 같이 현재 최적해와 직전에 발견한 최적해의 차이가 임계값(threshold value) 이하인 경우 수렴한 것으로 판단하는 방법이 있다.

$\text{if } |x^{(t+1)} - x^{(t)}| < \tau \text{ then break;}$

- 이를 Python 함수로 구현하면 다음과 같다.

```
# Check wheter the optimal solution converges
def is_converged(x_prev, x, thresold):
    if (abs(x_prev - x) < thresold):
        return True
    else:
        return False
```

④ 경사 하강법 알고리즘의 구현

- 앞서 ① ~ ③ 단계에서 구현한 함수들을 활용하여 경사 하강법 알고리즘을 구현하면 다음과 같다.
 - ✓ 이때, 각 시점(t)에서 발견된 최적해를 화면에 출력하는 코드도 포함하여 최적해가 어떻게 업데이트되는지 확인해보자.

```
# Gradient descent algorithm
def gradient_descent(f):
    lr = 0.01 # learning rate
    x = 100;  # Initialize the solution
    threshold = 1e-8
    t = 1

    while(True):
        x_prev = x # previous solution

        diff = fdiff(f)
        g = diff(x)

        x = x - lr * g # Update the optimal solution
        t += 1

        print(x)
        if(is_converged(x_prev, x, threshold)):
            break

    return x
```

⑤ 테스트



```
[31]: gradient_descent(f)
```

```
5.000000346655398  
5.0000003260052495  
5.000000306465324  
5.000000288035622  
5.000000270716143  
5.000000254506887  
5.000000239185809  
5.000000224974954  
5.000000211430233  
5.000000198773691  
5.000000187005327  
5.0000001759030965  
5.000000165467  
5.000000155474993
```

```
[31]: 5.000000155474993
```

Q & A